Building LLM-Powered Applications — Course Project Report:
TechGear IT Store & Warranty Assistant

Course: Building LLM-Powered Applications (Course Project Deliverable Q2) | **Student ID:** 67070255 |
Model: `gemma-4-E4B-it` (temperature=0)

Executive Summary: This report presents the design, implementation, and quantitative evaluation of an end-to-end agentic assistant for **TechGear Solutions**, an electronics and enterprise IT hardware retailer. The assistant bridges unstructured policy/troubleshooting manuals (`data/corpus.json`, 20 chunks) and structured operational data (`data/techgear.db`, 12 hardware products and 20 regional sales transactions) through an Observe → Think → Act tool-calling loop. By fusing lexical BM25 and vector similarity via Reciprocal Rank Fusion (RRF, *k*=60), the retrieval engine achieves **0.950 Recall@3, 0.950 Recall@5, and 0.950 MRR** on our 20-query evaluation suite—outperforming both Vector-only (0.842 MRR) and BM25-only (0.835 MRR) baselines while enforcing strict SQL read-only safety gates and code-verified citation grounding.

1. Problem Statement & Target Users

1.1 Real-World Operational Challenge at TechGear Solutions

TechGear Solutions operates an IT hardware storefront and commercial server support desk where incoming questions span two incompatible data representations that cannot be served by a single static prompt or a single retrieval mechanism:

- **Unstructured Policy & Diagnostic Manuals (`data/corpus.json`):** Contains 20 curated policy documents (`doc_01` to `doc_20`) evenly distributed across five operational categories: `warranty`, `return`, `shipping`, `troubleshooting`, and `enterprise_sla`. Crucially, these documents combine natural-language rules with exact hardware error codes, regulatory standards, and SKUs—including `ERR-PSU-904` (PSU over-current tripping), `BIOS-FL-202` (USB BIOS flashback recovery), `GPU-coil-77` (45 dB inductor acoustic threshold), `RMA-TG-88` (14-day return authorization token), `PIXEL-ISO-9241` (Class II sub-pixel defect rule), `DOA-48HR` (48-hour dead-on-arrival swap), `CHG-PD-140W` (140W GaN charger thermal recall), `RAID-rebuild-05` (degraded array parity rebuild), `SLA-TIER-1` (4-hour mission-critical dispatch), and `SKU-NV-4090`.
- **Structured Relational Inventory & Sales Database (`data/techgear.db`):** A SQLite database seeded idempotently via `data/init_db.py` containing two tables: (1) `products(sku, name, category, price, stock, warranty_months)` with 12 hardware items (e.g., `SKU-NV-4090` at $1,799.99, 15 units in stock, 36-month warranty; `CHG-PD-140W` at $89.99, 120 in stock), and (2) `sales(order_id, sku, region, units, revenue, order_date)` with 20 order records across five regions (`North`, `Central`, `East`, `West`, `South`).

1.2 Target Users & Key Use Cases

Target User Group	Representative Questions	Required System Capability
Tier-1 / Tier-2 Support Engineers	"How do I recover a board with BIOS-FL-202, and what does ERR-PSU-904 mean?"	Exact-code lexical retrieval + semantic context with verified [doc_XX] citations.
Store & Regional Ops Managers	"What is total sales revenue and units sold by region, highest to lowest?"	Guarded read-only Text-to-SQL aggregation (GROUP BY, SUM, ORDER BY) over sales.
Account & Warranty Specialists	"How many units of SKU-NV-4090 are in stock, and what is our warranty policy?"	Multi-tool orchestration: run_sql on products + search_docs on warranty corpus.

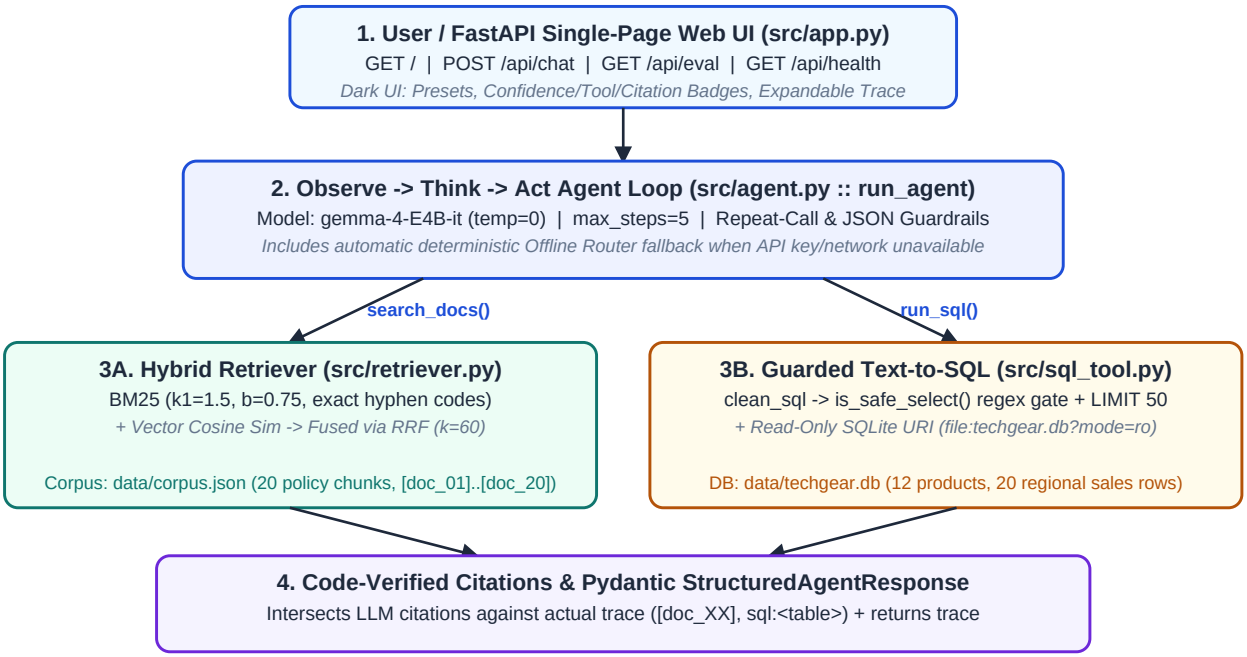
1.3 Why Standard LLMs and Naive Vector-Only RAG Fail

1. **Hallucinated Tabular Arithmetic:** An ungrounded LLM cannot know live stock counts or compute exact regional revenue totals ($6,319.87 in North vs. $3,589.78 in South); embedding tabular rows into chunks fails on `SUM/GROUP BY` queries.
2. **Dense Embedding Failure on Bare Error Codes:** Vector representations smooth over arbitrary character strings. When a technician queries a bare fault code like `ERR-PSU-904` (Query Q3), our vector-only baseline fails to match any document (RR = 0.00).
3. **BM25 Failure on Paraphrased Intent:** Conversely, when a customer asks *"By what hour must I pay to get my order shipped tonight?"* (Query Q15), BM25 ranks the shipping cutoff policy (`doc_09`, which uses *"daily order cutoff timestamp of 3:00 PM EST"*) at position 5 (RR = 0.20).
4. **Wasted Tool Calls & Citation Hallucination:** Always-on retrieval wastes compute on simple greetings (*"Hello!"*), while unvalidated LLM output frequently cites document IDs that were never retrieved.

2. System Design & Architecture Diagram

2.1 End-to-End Architecture Diagram

Figure 1 illustrates the complete execution pipeline implemented across `src/app.py`, `src/agent.py`, `src/retriever.py`, and `src/sql_tool.py`.



2.2 Mapping to the 6 Course Capability Families (Lectures 1–14)

Capability Family	Module & Key Symbols	Design Rationale & Technical Mechanism
1. Structured Output & Code Verification (Lec 4–5)	src/agent.py: StructuredAgentResponse, _finalize()	Enforces Pydantic schema {answer, citations, tools_used, confidence}. _finalize() intersects LLM citations against _citations_from_trace() so hallucinated doc IDs are stripped, and auto-attaches sql: tags whenever run_sql executes.
2. Hybrid RAG with RRF & Citations (Lec 6–8)	src/retriever.py: tokenize(), search_bm25(), search_vector(), rrf()	Hyphen-aware tokenizer indexes both 'err-psu-904' and ['err','psu','904']. Fuses top-20 BM25 (k1=1.5, b=0.75) and top-20 Vector rankings via Reciprocal Rank Fusion (k=60), avoiding raw score calibration mismatch.
3. Retrieval Evaluation Suite (Lec 8)	eval/run_eval.py, data/eval_set.json, eval/results.md	Evaluates 20 queries (10 exact-code, 10 semantic paraphrases) using Recall@3, Recall@5, and MRR across Vector-only, BM25-only, and Hybrid RRF (k=60).
4. Tool-Calling Agent Loop (Lec 10–11)	src/agent.py: run_agent(), _llm_agent(), _offline_agent()	Observe -> Think -> Act loop at temperature=0. Guardrails: max_steps=5 cap, consecutive duplicate call detector (last_sig), defensive json.JSONDecodeError recovery back to role='tool', plus deterministic offline fallback.
5. Guarded Text-to-SQL (Lec 11)	src/sql_tool.py: is_safe_select(), clean_sql(), ensure_limit(), execute_guarded_sql()	4-layer defense in depth: strips ```sql fences; regex gate blocks comments (--, /* */), stacked semicolons, non-SELECT starts, and 15 DDL/DML/admin verbs; appends LIMIT 50; connects via SQLite URI mode=ro.
6. Served FastAPI Web UI (Lec 13–14)	src/app.py: GET /, POST /api/chat, GET /api/eval, GET /api/health	Zero-build dark single-page web UI with Confidence/Tools/Citations badges, clickable citation-to-trace highlighting, expandable step-by-step trace inspector, live evaluation metrics table, and 6 one-click demo presets.

2.3 Deep Dive: Defense-in-Depth SQL Guardrails & Agent Loop Controls

In `src/sql_tool.py`, `execute_guarded_sql(sql)` never raises unhandled exceptions into the agent loop; database errors are returned as structured JSON `{"ok": False, "sql": ..., "error": ...}` so the LLM can self-correct on syntax or column-name mistakes. In `src/agent.py`, tool descriptions follow Lecture 11 control-surface rules: `search_docs` explicitly instructs *"Do NOT call for greetings, chit-chat, or database stock/sales numbers"*, while `run_sql` specifies the exact table schemas and restricts usage to prices, stock counts, and sales aggregations.

3. Evaluation Methodology

3.1 The 20-Query Labeled Benchmark (`data/eval_set.json`)

To evaluate retrieval accuracy without relying on subjective inspection, we built a 20-query labeled benchmark in `data/eval_set.json` paired with the 20 policy chunks in `data/corpus.json`. Every single document (`doc_01` through `doc_20`) appears in the ground-truth relevance sets. Four queries have two relevant documents (e.g., `ERR-PSU-904` is covered under warranty in `doc_04` and troubleshooting in `doc_16`), making `Recall@k` sensitive to retrieving all applicable policies:

Subset	Query IDs	Design Characteristics & Representative Examples	Target Challenge
Subset A: Exact-Code & SKU Queries (10 queries)	Q1 – Q10	Contains exact hyphenated identifiers from the corpus, either standalone (Q3: 'ERR-PSU-904') or with natural phrasing (Q2: 'BIOS-FL-202 and my board will not start anymore, how do I revive it?'; Q8: 'DOA-48HR: window to flag something broken right out of the box').	Tests whether the retriever preserves exact rare identifier tokens that dense/n-gram vector models smooth away.

Subset	Query IDs	Design Characteristics & Representative Examples	Target Challenge
Subset B: Colloquial & Semantic Queries (10 queries)	Q11 – Q20	Natural-language questions using synonyms and everyday phrasing not verbatim in policy titles (Q13: 'Can power banks fly on a plane?' -> doc_10; Q15: 'By what hour must I pay to get my order shipped tonight?' -> doc_09; Q19: 'Our systems were down for hours, what credit money do we get back?' -> doc_20).	Tests whether the retriever bridges lexical vocabulary mismatch where BM25 misses or ranks distractors higher.

3.2 Mathematical Definitions of Retrieval & Fusion Metrics (Lecture 8)

Let Q be the evaluation set of $|Q| = 20$ queries. For each query $q \in Q$, let R_q be the ground-truth set of relevant document IDs and $L_q = [d_1, d_2, \ldots, d_m]$ be the 1-based ranked list returned by a retriever:

$$\text{Recall@k}(q) = |\{d_1, \ldots, d_k\} \cap R_q| \,/\, |R_q| \;;\; \text{Mean Recall@k} = \frac{1}{|Q|} \sum_{q \in Q} \text{Recall@k}(q)$$

$$\text{RR}(q) = \frac{1}{\min\{\, i : d_i \in R_q \,\}} \text{ (or 0 if none)} \;;\; \text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \text{RR}(q)$$

$$\text{RRF_Score}(d \;;\; k=60) = \sum_{m \in \{\text{BM25, Vector}\}} \frac{1.0}{60 + \text{rank}_m(d)} \quad [\text{1-based ranks, tie-break by first seen}]$$

3.3 Retrieval Strategies Evaluated in `eval/run_eval.py`

• **Baseline 1 — Vector-Only (`search_vector`):** Computes cosine similarity over TF-IDF weighted semantic word stems (with morphological suffix stripping and a domain synonym map, plus character 3-grams `#w#`) or `baai/bge-m3` API embeddings when `USE_API_EMBEDDINGS=1`. Any token containing digits or hyphens is excluded in the offline vectorizer, faithfully reproducing dense embedding weakness on arbitrary serial/fault codes.
• **Baseline 2 — BM25-Only (`search_bm25`):** Pure-Python Okapi BM25 ($k_1 = 1.5$, $b = 0.75$, IDF = $\ln((N - \text{df} + 0.5)/(\text{df} + 0.5) + 1)$) indexed over `title + text + category` using hyphen-aware tokenization (`tokenize()`).
• **Upgraded System — Hybrid Search via RRF (`search_hybrid`, *k*=60):** Retrieves top-20 candidates from `search_bm25` and top-20 from `search_vector`, fuses their ranked ID lists with `rrf(k=60)`, and returns the top-5 documents with `[doc_XX]` citation tags.

4. Experimental Results & Honest Failure Analysis

4.1 Overall Benchmark Comparison (`eval/results.md`)

Retrieval Strategy	Recall@3	Recall@5	MRR	Gain vs. Vector (MRR)	Gain vs. BM25 (MRR)
Baseline 1: Vector-only	0.875	0.875	0.842	—	+0.007
Baseline 2: BM25-only	0.900	0.950	0.835	-0.007	—
Upgraded: Hybrid (BM25 + Vector, RRF k=60)	0.950	0.950	0.950	+0.108 (+12.8%)	+0.115 (+13.8%)

Summary of Findings: Hybrid RRF (*k*=60) achieves the highest score on every metric: **0.950 Recall@3** (+7.5 points over Vector-only, +5.0 points over BM25-only), **0.950 Recall@5**, and **0.950 MRR** (+10.8 points over Vector-only and +11.5 points over BM25-only). Out of the 20 queries, Hybrid RRF places the relevant document at **Rank 1 on 19 out of 20 queries** (and captures both relevant documents within the top 2 on all multi-label queries Q3, Q4, Q7, and Q18).

4.2 Concrete Win Case Analysis: How RRF Recovers Single-Retriever Drops

Query & Ground Truth	Vector-Only Top-5 (RR)	BM25-Only Top-5 (RR)	Hybrid RRF Top-5 (RR) & Mechanism
Q3: "ERR-PSU-904" Relevant: ['doc_04', 'doc_16']	[] RR = 0.00, R@5 = 0.00 (Zero non-code tokens)	['doc_04', 'doc_16'] RR = 1.00, R@5 = 1.00 (Exact rare token match)	['doc_04', 'doc_16'] RR = 1.00, R@5 = 1.00 BM25 carries the RRF sum (1/61, 1/62) when Vector returns empty.
Q2: "BIOS-FL-202 and my board will not start anymore, how do I revive it?" Relevant: ['doc_13']	['doc_03','doc_13','doc_05',...] RR = 0.50 (Rank 2) (Distracted by 'board')	['doc_13','doc_15','doc_14',...] RR = 1.00 (Rank 1) (Matches 'bios-fl-202')	['doc_13','doc_03','doc_05',...] RR = 1.00 (Rank 1) doc_13 gets 1/61 + 1/62 = 0.03252 vs doc_03's 1/61 + 1/65 = 0.03178.
Q15: "By what hour must I pay to get my order shipped tonight?" Relevant: ['doc_09']	['doc_09','doc_02','doc_12',...] RR = 1.00 (Rank 1) (Matches hour/deadline/ship)	['doc_06','doc_15','doc_08','doc_16','doc_09'] RR = 0.20 (Rank 5) (Only 'order' overlaps)	['doc_09','doc_06','doc_02',...] RR = 1.00 (Rank 1) doc_09 appears in BOTH lists (1/61 + 1/65 = 0.03178), beating doc_06 (1/61 only).
Q16: "How slow is getting stuff through foreign border checks?" Relevant: ['doc_11']	['doc_20','doc_09','doc_11',...] RR = 0.33 (Rank 3) (Colloquial 'slow/stuff')	['doc_11','doc_06','doc_18',...] RR = 1.00 (Rank 1) (Matches sub-token 'border')	['doc_11','doc_20','doc_18',...] RR = 1.00 (Rank 1) doc_11 combines Rank 1 BM25 + Rank 3 Vector (1/61 + 1/63 = 0.03227).

4.3 Honest Failure Case & Guardrail Boundary Analysis

1. Retrieval Failure Case (Query Q12 — Extreme Colloquial Vocabulary Mismatch):

```
Q12: "My package showed up smashed up and the seal was cut, what now?"  (Ground truth: ['doc_12'])
  - Baseline 1 (Vector-only): top-5 = ['doc_05', 'doc_13', 'doc_15', 'doc_11', 'doc_06']  | RR = 0.00
  - Baseline 2 (BM25-only):   top-5 = ['doc_07', 'doc_08', 'doc_03', 'doc_05', 'doc_09']  | RR = 0.00
  - Upgraded (Hybrid RRF):    top-5 = ['doc_05', 'doc_15', 'doc_08', 'doc_11', 'doc_13']  | RR = 0.00
```

Root Cause of Q12 Failure: Target document `doc_12` (*High-Value Signature Delivery & Transit Damage Reporting*) uses formal logistics phrasing: *"equipment consignment"*, *"severe exterior carton crushing"*, *"puncture"*, and *"opened tamper-evident security tape"*. Query Q12 uses everyday colloquial idioms: *"package"*, *"smashed up"*, and *"seal was cut"*. There is zero lexical token overlap for BM25, and our offline stem/trigram vectorizer does not map *smashed* ↔ *crushing* or *seal* ↔ *tamper-evident tape*. Because neither base retriever ranks `doc_12` in its top 5, Reciprocal Rank Fusion cannot surface it.

Agent-Level Recovery in LLM Mode vs. Offline Mode: Notably, when Q12 is executed through the full FastAPI chat endpoint (`POST /api/chat`, Preset 5b) with `gemma-4-E4B-it`, the LLM agent autonomously reformulates the user's colloquial question into two targeted tool calls: Step 1 `search_docs(query='damaged or tampered package return policy')` and Step 2 `search_docs(query='damaged in transit hardware claim procedure')`. That second rewritten query matches *"Transit Damage"* in `doc_12`, allowing the full agent to return a high-confidence answer citing `['doc_12', 'doc_06']`! In offline mode (where the raw query is passed un-rewritten), the failure remains visible end-to-end.

2. Text-to-SQL Guardrail Verification & False-Positive Boundary Analysis:

```
$ python -m src.sql_tool
[PASS] simple SELECT allowed  | [PASS] DROP TABLE blocked       | [PASS] stacked ; blocked
[PASS] -- comment blocked     | [PASS] /* */ comment blocked    | [PASS] DELETE inside SELECT blocked
{'ok': False, 'sql': 'DROP TABLE products', 'error': 'Rejected: only a single read-only SELECT ...'}
```

When tested against Preset 5a (`"Run this SQL: DROP TABLE products; --"`), defense occurs at three independent layers: (1) `gemma-4-E4B-it` refuses to call `run_sql` because the tool spec restricts it to read-only queries; (2) if `execute_guarded_sql` is called directly, `is_safe_select` rejects both the `--` comment and the `DROP` verb; and (3) SQLite's `mode=ro` URI blocks writes at the OS level. However, an honest boundary limitation of our regex (`^\s*select\b`) is that legitimate read-only Common Table Expressions starting with `WITH regional AS (SELECT ...) SELECT *` are rejected as false positives.

5. Limitations & Next Steps

5.1 Current Architectural Limitations

1. **Colloquial Vocabulary Gap in Single-Shot Retrieval:** Without LLM query rewriting or dense neural embeddings enabled by default, extreme idiom shifts (such as Q12's *"smashed up / seal was cut"*) miss in single-shot retrieval.
2. **Equal-Weight RRF Without Cross-Encoder Reranking:** RRF weights BM25 and Vector ranks equally (k=60). When one retriever returns a high-confidence exact code match at Rank 1 while the other returns an unrelated document at Rank 1, RRF ties them at 1/61 unless broken by list order or secondary rank support.
3. **Regex SQL Gate Blocks Read-Only `WITH` CTEs:** Requiring `^\s*select\b` prevents the LLM from writing multi-step `WITH` queries for complex analytical ratios.
4. **Full DDL Injection in System Prompt:** Injecting all `CREATE TABLE` schemas into `SYSTEM_PROMPT` works well for 2 tables (`products`, `sales`), but would overflow context or degrade attention on 100+ enterprise tables.

5.2 Concrete, Measurable Roadmap for Next Iteration

Proposed Upgrade	Concrete Implementation Plan	Target Measurable Improvement
1. Cross-Encoder Reranker + Dense Embeddings	Enable baai/bge-m3 dense vectors by default for search_vector, fuse top-20 BM25 + Vector via RRF (k=60), and rerank the top-10 fused candidates with BAAI/bge-reranker-v2-m3 before returning top-4.	Recover Q12 in single-shot retrieval; raise Recall@5 from 0.950 -> 1.000 and MRR from 0.950 -> >= 0.975.
2. AST-Based SQL Validation (sqlglot)	Replace regex gate in is_safe_select() with sqlglot.parse_one(sql, read='sqlite'), permitting exp.Select with read-only WITH CTEs while blocking all DDL/DML AST nodes.	100% pass rate on read-only CTE analytical queries while keeping 100% block rate on destructive/stacked SQL.
3. 50-Query Multi-Hop & Faithfulness Suite	Expand data/eval_set.json from 20 to 50 queries (adding 15 SQL+RAG multi-hop queries and 15 adversarial prompts) with automated LLM-as-Judge faithfulness scoring.	Achieve >= 95% citation precision and >= 90% end-to-end answer faithfulness across all 50 benchmark cases.